

PL/SQL Complete Guide

A comprehensive beginner-to-advanced guide for learning Oracle PL/SQL.

Table of Contents

1. Introduction to PL/SQL
2. PL/SQL Architecture
3. Structure of a PL/SQL Block
4. Variables and Constants
5. Data Types
6. Operators
7. Conditional Statements
8. Loops
9. Exception Handling
10. Cursors
11. Procedures
12. Functions
13. Packages
14. Triggers
15. Dynamic SQL
16. Collections
17. Records
18. Bulk Processing
19. File Handling
20. Transactions
21. Performance Tuning
22. Best Practices
23. Real World Examples
24. Interview Questions
25. Conclusion

Introduction to PL/SQL

What is PL/SQL?

PL/SQL stands for:

Procedural Language / Structured Query Language

It is Oracle's procedural extension of SQL.

PL/SQL combines:

- SQL capabilities
 - Procedural programming
 - Business logic
 - Exception handling
-

Features of PL/SQL

- Tight integration with SQL
 - Block-structured language
 - Supports variables and constants
 - Supports loops and conditions
 - Exception handling
 - High performance
 - Modular programming
 - Secure database programming
-

PL/SQL Architecture

PL/SQL engine works with Oracle SQL engine.

```
graph TD; A[Application] --> B[PL/SQL Engine]; B --> C[SQL Engine]; C --> D[Oracle Database];
```

Application
↓
PL/SQL Engine
↓
SQL Engine
↓
Oracle Database

Structure of a PL/SQL Block

A PL/SQL block contains:

1. Declaration Section
 2. Executable Section
 3. Exception Section
-

Basic Syntax

```
DECLARE
    -- Variable declarations
BEGIN
    -- Executable statements
EXCEPTION
    -- Error handling
END;
/
```

First PL/SQL Program

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello World');
END;
/
```

Variables and Constants

Variable Declaration

```
DECLARE
    v_name VARCHAR2(100);
    v_age NUMBER;
BEGIN
    v_name := 'John';
    v_age := 25;

    DBMS_OUTPUT.PUT_LINE(v_name);
    DBMS_OUTPUT.PUT_LINE(v_age);
END;
/
```

Constants

```
DECLARE
    c_tax CONSTANT NUMBER := 16;
BEGIN
    DBMS_OUTPUT.PUT_LINE(c_tax);
```

```
END;  
/
```

Data Types

Character Types

Type	Description
CHAR	Fixed-length string
VARCHAR2	Variable-length string
CLOB	Large text

Numeric Types

Type	Description
NUMBER	Numeric values
PLS_INTEGER	Integer
BINARY_FLOAT	Floating-point

Date Types

Type	Description
DATE	Date and time
TIMESTAMP	Precise timestamp

Operators

Arithmetic Operators

Operator	Example
+	a + b
-	a - b
*	a * b

Operator	Example
/	a / b

Comparison Operators

Operator	Meaning
=	Equal
!=	Not Equal
>	Greater Than
<	Less Than

Conditional Statements

IF Statement

```
DECLARE
    v_marks NUMBER := 80;
BEGIN

    IF v_marks >= 50 THEN
        DBMS_OUTPUT.PUT_LINE('PASS');
    END IF;

END;
/
```

IF ELSE

```
DECLARE
    v_marks NUMBER := 40;
BEGIN

    IF v_marks >= 50 THEN
        DBMS_OUTPUT.PUT_LINE('PASS');
    ELSE
        DBMS_OUTPUT.PUT_LINE('FAIL');
    END IF;

END;
```

```
END;  
/  

```

ELSIF

```
DECLARE  
    v_marks NUMBER := 75;  
BEGIN  
  
    IF v_marks >= 80 THEN  
        DBMS_OUTPUT.PUT_LINE('A');  
  
    ELSIF v_marks >= 60 THEN  
        DBMS_OUTPUT.PUT_LINE('B');  
  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('C');  
    END IF;  
  
END;  
/  

```

CASE Statement

```
DECLARE  
    v_grade CHAR := 'A';  
BEGIN  
  
    CASE v_grade  
        WHEN 'A' THEN  
            DBMS_OUTPUT.PUT_LINE('Excellent');  
  
        WHEN 'B' THEN  
            DBMS_OUTPUT.PUT_LINE('Good');  
  
        ELSE  
            DBMS_OUTPUT.PUT_LINE('Average');  
    END CASE;  
  
END;  
/  

```

Loops

LOOP

```
DECLARE
    v_counter NUMBER := 1;
BEGIN

    LOOP
        DBMS_OUTPUT.PUT_LINE(v_counter);

        v_counter := v_counter + 1;

        EXIT WHEN v_counter > 5;
    END LOOP;

END;
/
```

WHILE LOOP

```
DECLARE
    v_counter NUMBER := 1;
BEGIN

    WHILE v_counter <= 5 LOOP

        DBMS_OUTPUT.PUT_LINE(v_counter);

        v_counter := v_counter + 1;

    END LOOP;

END;
/
```

FOR LOOP

```
BEGIN

    FOR i IN 1..5 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;

END;
/
```

Exception Handling

Basic Exception Block

```
BEGIN

    DBMS_OUTPUT.PUT_LINE(10 / 0);

EXCEPTION

    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');

END;
/
```

Common Exceptions

Exception	Meaning
NO_DATA_FOUND	No row returned
TOO_MANY_ROWS	Multiple rows returned
ZERO_DIVIDE	Divide by zero
INVALID_NUMBER	Invalid numeric conversion

Cursors

Cursors process query results row by row.

Explicit Cursor Example

```
DECLARE

    CURSOR c_employees IS
        SELECT employee_id, first_name
        FROM employees;

    v_id employees.employee_id%TYPE;
    v_name employees.first_name%TYPE;

BEGIN

    OPEN c_employees;

    LOOP

        FETCH c_employees INTO v_id, v_name;

        EXIT WHEN c_employees%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(v_id || ' - ' || v_name);

    END LOOP;

    CLOSE c_employees;

END;
/
```

Cursor FOR LOOP

```
BEGIN

    FOR rec IN (
        SELECT employee_id, first_name
        FROM employees
    ) LOOP

        DBMS_OUTPUT.PUT_LINE(
            rec.employee_id || ' - ' || rec.first_name
        );

    END LOOP;
```

```
END;  
/
```

Procedures

Procedures are reusable program units.

Creating a Procedure

```
CREATE OR REPLACE PROCEDURE hello_world  
IS  
BEGIN  
  
    DBMS_OUTPUT.PUT_LINE('Hello World');  
  
END;  
/
```

Procedure with Parameters

```
CREATE OR REPLACE PROCEDURE add_numbers(  
    p_num1 NUMBER,  
    p_num2 NUMBER  
)  
IS  
    v_total NUMBER;  
BEGIN  
  
    v_total := p_num1 + p_num2;  
  
    DBMS_OUTPUT.PUT_LINE(v_total);  
  
END;  
/
```

Calling a Procedure

```
BEGIN
    add_numbers(10, 20);
END;
/
```

Functions

Functions return values.

Creating a Function

```
CREATE OR REPLACE FUNCTION calculate_tax(
    p_amount NUMBER
)
RETURN NUMBER
IS
BEGIN

    RETURN p_amount * 0.16;

END;
/
```

Calling a Function

```
DECLARE
    v_tax NUMBER;
BEGIN

    v_tax := calculate_tax(1000);

    DBMS_OUTPUT.PUT_LINE(v_tax);

END;
/
```

Packages

Packages group related procedures and functions.

Package Specification

```
CREATE OR REPLACE PACKAGE pkg_employee
IS

    PROCEDURE add_employee;

    FUNCTION get_salary(
        p_emp_id NUMBER
    ) RETURN NUMBER;

END pkg_employee;
/
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY pkg_employee
IS

    PROCEDURE add_employee
    IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Employee Added');
    END;

    FUNCTION get_salary(
        p_emp_id NUMBER
    ) RETURN NUMBER
    IS
    BEGIN
        RETURN 50000;
    END;

END pkg_employee;
/
```

Triggers

Triggers execute automatically.

BEFORE INSERT Trigger

```
CREATE OR REPLACE TRIGGER trg_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN

    :NEW.created_date := SYSDATE;

END;
/
```

AFTER UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_after_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN

    INSERT INTO audit_log(
        table_name,
        updated_date
    )
    VALUES(
        'EMPLOYEES',
        SYSDATE
    );

END;
/
```

Dynamic SQL

Dynamic SQL allows runtime query execution.

EXECUTE IMMEDIATE Example

```
DECLARE
    v_sql VARCHAR2(1000);
BEGIN

    v_sql := 'DELETE FROM employees WHERE employee_id = 100';

    EXECUTE IMMEDIATE v_sql;

END;
/
```

Records

Records store multiple related fields.

Record Example

```
DECLARE

    TYPE emp_record IS RECORD(
        employee_id NUMBER,
        first_name VARCHAR2(100)
    );

    v_emp emp_record;

BEGIN

    v_emp.employee_id := 1;
    v_emp.first_name := 'John';

    DBMS_OUTPUT.PUT_LINE(v_emp.first_name);

END;
/
```

Collections

Collections store multiple values.

Associative Array Example

```
DECLARE

    TYPE t_names IS TABLE OF VARCHAR2(100)
    INDEX BY PLS_INTEGER;

    v_names t_names;

BEGIN

    v_names(1) := 'John';
    v_names(2) := 'Mary';

    DBMS_OUTPUT.PUT_LINE(v_names(1));

END;
/
```

Bulk Processing

Bulk processing improves performance.

BULK COLLECT

```
DECLARE

    TYPE t_ids IS TABLE OF employees.employee_id%TYPE;

    v_ids t_ids;

BEGIN

    SELECT employee_id
    BULK COLLECT INTO v_ids
    FROM employees;

END;
/
```

FORALL

```
DECLARE

    TYPE t_ids IS TABLE OF NUMBER;

    v_ids t_ids := t_ids(1,2,3);

BEGIN

    FORALL i IN 1..v_ids.COUNT

        DELETE FROM employees
        WHERE employee_id = v_ids(i);

END;
/
```

File Handling

UTL_FILE package handles files.

File Write Example

```
DECLARE

    v_file UTL_FILE.FILE_TYPE;

BEGIN

    v_file := UTL_FILE.FOPEN(
        'REPORT_DIR',
        'sample.txt',
        'W'
    );

    UTL_FILE.PUT_LINE(v_file, 'Hello PL/SQL');

    UTL_FILE.FCLOSE(v_file);

END;
/
```


Transactions

Transactions maintain database consistency.

COMMIT

```
COMMIT;
```

ROLLBACK

```
ROLLBACK;
```

SAVEPOINT

```
SAVEPOINT before_update;
```

Performance Tuning

Best Performance Tips

- Use BULK COLLECT
 - Use FORALL
 - Avoid row-by-row processing
 - Use indexes properly
 - Minimize context switching
 - Use bind variables
-

Best Practices

Coding Standards

- Use meaningful variable names
- Handle exceptions properly
- Modularize code using packages

- Avoid hardcoded values
 - Use comments wisely
-

Real World Example

Employee Salary Update Procedure

```
CREATE OR REPLACE PROCEDURE update_salary(  
    p_emp_id NUMBER,  
    p_increment NUMBER  
)  
IS  
BEGIN  
  
    UPDATE employees  
    SET salary = salary + p_increment  
    WHERE employee_id = p_emp_id;  
  
    COMMIT;  
  
    DBMS_OUTPUT.PUT_LINE('Salary Updated');  
  
EXCEPTION  
  
    WHEN OTHERS THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE(SQLERRM);  
  
END;  
/
```

Interview Questions

Common PL/SQL Interview Questions

1. Difference between Procedure and Function?
2. What is a Cursor?
3. Difference between Implicit and Explicit Cursor?
4. What is BULK COLLECT?
5. What is FORALL?
6. What are Triggers?
7. Difference between Package Spec and Body?
8. What is Dynamic SQL?
9. Explain Exception Handling.

Key Concepts Summary

Concept	Purpose
Procedure	Reusable business logic
Function	Returns a value
Trigger	Auto-executed logic
Package	Group related programs
Cursor	Process rows
Collection	Store multiple values
Exception	Error handling
Dynamic SQL	Runtime SQL execution

Conclusion

PL/SQL is a powerful enterprise database programming language.

Mastering PL/SQL helps you build:

- Enterprise Systems
- Banking Applications
- Insurance Systems
- ERP Systems
- Financial Applications
- Oracle ADF Applications
- Reporting Engines

Advanced PL/SQL knowledge is critical for Oracle developers working with:

- Oracle Forms
- Oracle ADF
- BI Publisher
- Stored Procedures
- Backend APIs
- High-performance database systems